Assignments - 10

Exercise 1: Navigation and Title Verification
Write a script that launches a browser and navigates to the Amazon home page. Once loaded, capture the page title and use an assertion to verify that it contains the word "Amazon". After the check is complete, navigate to the "Mobiles" category page and go back to the home page using browser navigation methods.

Exercise 2: Basic Locators and Search
Identify the Amazon search bar using its `id` and the search button using its `xpath`. Type "Wireless Headphones" into the search bar and click the search button. Verify that the results page displayed contains a specific header or text indicating that results for your search are being shown.

Exercise 3: Implementing Implicit and Explicit Waits
Create a test that searches for a specific laptop model. Implement an Implicit Wait at the driver level to handle general loading. Then, use an Explicit Wait (`WebDriverWait`) with an `ExpectedCondition` to wait specifically for the "Results" grid or a specific product image to become visible before attempting to click the first result.

Exercise 4: Advanced Locators (CSS Selectors & Links)
Navigate to the Amazon footer section. Use CSS Selectors to identify and click on the "About Us" link. On the following page, find a specific element using its `name` or `link_text` and print its text content to the console.

Exercise 5: Element Interaction and Synchronization
Automate the process of filtering search results. Search for "Smart Watches," then locate the "Brand" filter in the left-hand sidebar. Click on a specific brand checkbox (e.g., "Samsung" or "Apple"). Because filters refresh the page asynchronously, implement a wait strategy to ensure the page has updated before counting how many products are displayed on the first page.

Folder Structure:

```
amazon_automation/
├── src/
│   └── amazon_auto.py       # Methods to interact with Amazon (Locators & Actions)
└── tests/
    └── test_amazon_auto.py      # Your actual exercise test cases
```

CODES:

amazon_auto.py

```
import time
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC


class AmazonPage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(self.driver, 15)

    def navigate_to_mobiles_and_back(self):
        mobile_link = self.driver.find_element(By.LINK_TEXT, "Mobiles")
        mobile_link.click()
        self.driver.back()

    def search_wireless_headphones(self):
        search_bar = self.driver.find_element(By.ID, "twotabsearchtextbox")
        search_button = self.driver.find_element(By.XPATH,
"//input[@id='nav-search-submit-button']")
        search_bar.clear()
        search_bar.send_keys("Wireless Headphones")
        search_button.click()
        header = self.driver.find_element(By.XPATH,
"//span[@class='a-color-state a-text-bold']")
        return header.text

    def search_laptop_and_click_first(self, model):
        search_bar = self.driver.find_element(By.ID, "twotabsearchtextbox")
        search_bar.clear()
        search_bar.send_keys(model + Keys.ENTER)
        first_result =
self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "img.s-image")))
        first_result.click()

    def click_about_us_and_get_header(self):
        # Using the exact CSS selector from Exercise 4 logic
        about_us = self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
"a[href*='www.aboutamazon']")))
        about_us.click()
        header = self.wait.until(EC.presence_of_element_located((By.TAG_NAME,
"h1")))
        return header.text

    def search_smart_watches_and_filter_samsung(self):
```

```
        search_bar = self.driver.find_element(By.ID, "twotabsearchtextbox")
        search_bar.clear()
        search_bar.send_keys("Smart Watches" + Keys.ENTER)

        # Exercise 5 logic: Handle "Show All" and filter
        show_all = self.wait.until(
            EC.element_to_be_clickable((By.XPATH,
"//*[@id='filter-p_123']/span/li/span/div/a/span")))
        show_all.click()
        brand_filter = self.wait.until(EC.element_to_be_clickable((By.XPATH,
"//span[text()='Samsung']")))
        brand_filter.click()

        time.sleep(3)  # Wait for async refresh
        products = self.driver.find_elements(By.XPATH,
"//div[@data-component-type='s-search-result']")
        return len(products)
```

test_amazon_auto.py

```
import unittest
import sys
import os
from selenium import webdriver

# Ensures the 'src' folder is reachable for imports
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
'..')))
from src.amazon_auto import AmazonPage

class TestAmazonAutomation(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        """Setup performed once before all tests."""
        cls.driver = webdriver.Edge()
        cls.driver.maximize_window()
        cls.driver.implicitly_wait(10)
        cls.amazon = AmazonPage(cls.driver)

    def setUp(self):
        """Navigate to home page before every individual test exercise."""
        self.driver.get("https://amazon.in")

    def test_exercise_1_navigation(self):
        print("\nRunning Exercise 1...")
        self.assertIn("Amazon", self.driver.title)
        self.amazon.navigate_to_mobiles_and_back()
```

```
    def test_exercise_2_search(self):
        print("\nRunning Exercise 2...")
        header_text = self.amazon.search_wireless_headphones()
        self.assertIn("Wireless Headphones", header_text)

    def test_exercise_3_explicit_wait(self):
        print("\nRunning Exercise 3...")
        # This will search and click the first result using Explicit Wait
        self.amazon.search_laptop_and_click_first("MacBook Pro")

    def test_exercise_4_footer_css(self):
        print("\nRunning Exercise 4...")
        header_text = self.amazon.click_about_us_and_get_header()
        print(f"Captured Header: {header_text}")
        self.assertTrue(len(header_text) > 0)

    def test_exercise_5_filter_sync(self):
        print("\nRunning Exercise 5...")
        count = self.amazon.search_smart_watches_and_filter_samsung()
        print(f"Found {count} Samsung products.")
        self.assertGreater(count, 0)

    @classmethod
    def tearDownClass(cls):
        """Close browser after all tests are finished."""
        cls.driver.quit()

# if __name__ == "__main__":
#     unittest.main()
```

Results:

(.venv) PS C:\Wipro Training\Assignments> python -m unittest -v test.test_amazon_auto.TestAmazonAutomation

DevTools listening on ws://127.0.0.1:54209/devtools/browser/84c7b949-aa16-4160-b617-0e2c9805553f
test_exercise_1_navigation (test.test_amazon_auto.TestAmazonAutomation.test_exercise_1_navigation) ...
[12208:12992:0511/020648.632:ERROR:chrome\browser\task_manager\providers\fallback_task_provider.cc:126] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.

Running Exercise 1...

[12208:12992:0511/020650.758:ERROR:chrome\browser\task_manager\providers\fallback_task_provider.cc:126] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.
[12208:12992:0511/020650.790:ERROR:chrome\browser\task_manager\providers\fallback_task_provider.cc:126] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.
ok
test_exercise_2_search
(test.test_amazon_auto.TestAmazonAutomation.test_exercise_2_search) ...
[12208:12992:0511/020655.572:ERROR:chrome\browser\task_manager\providers\fallback_task_provider.cc:126] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.

Running Exercise 2...
[12208:12992:0511/020657.684:ERROR:chrome\browser\task_manager\providers\fallback_task_provider.cc:126] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.
ok
test_exercise_3_explicit_wait
(test.test_amazon_auto.TestAmazonAutomation.test_exercise_3_explicit_wait) ...
Running Exercise 3...
ok
test_exercise_4_footer_css
(test.test_amazon_auto.TestAmazonAutomation.test_exercise_4_footer_css) ...
[12208:12992:0511/020703.504:ERROR:chrome\browser\task_manager\providers\fallback_task_provider.cc:126] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.

Running Exercise 4...
[12208:12992:0511/020707.290:ERROR:chrome\browser\task_manager\providers\fallback_task_provider.cc:126] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.
Captured Header: Prime Video becomes India’s largest streaming service for exclusive originals as Amazon MX Player joins in
ok
test_exercise_5_filter_sync
(test.test_amazon_auto.TestAmazonAutomation.test_exercise_5_filter_sync) ...
Running Exercise 5...

Found 16 Samsung products.
ok

Ran 5 tests in 37.447s

OK